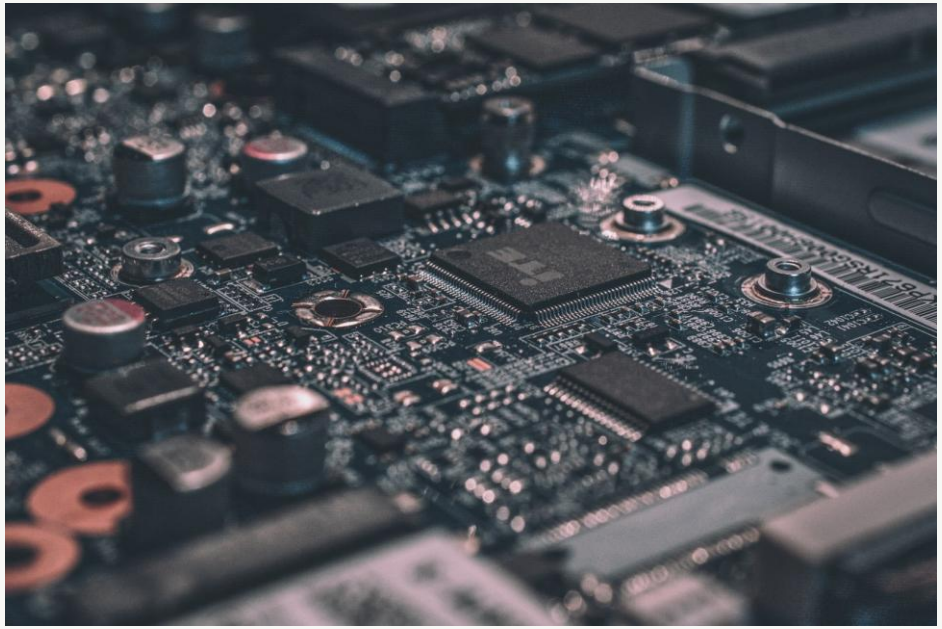
Miért nem mindig pontos a 0,1 + 0,2?

Lebegőpontos számábrázolás

IEEE 754, pontosság, kerekítés
és a számítási teljesítmény



Nagy tartomány, véges pontosság.

Egy első meglepetés

$$0{,}1 + 0{,}2 \neq 0{,}3?$$

A matematikában egyenlő. A tárolt bináris közelítésekkel végzett számítás eltérhet.

Három kérdés vezeti végig az órát

Hogyan tároljuk?

Előjel.
Karakterisztika.
Szignifikandusz.

Egy szám kódolása bitekkel.

Mennyire pontos?

Véges számú helyiérték.

Kerekítés, nagy és nagyon kis számok.

Milyen gyorsan?

Lebegőpontos műveletek.

FLOPS, pontosság és mért teljesítmény.

Előbb megértjük a számot, utána a formátumot és a műveletet.

Egész, fixpontos, lebegőpontos

Egész

A számnak nincs külön tört része.

Példa: 125 darab.

Fixpontos

A skála előre rögzített.

Példa: 1250 egység jelentése 12,50.

Lebegőpontos

A nagyságrendet a kitevő állítja be.

Példa: $1{,}25 \times 10^3$.

A „lebegő” elnevezés a radixpont helyének változó értelmezésére utal.

Ismerős ötlet: tudományos jelölés

Nagy szám	$12\,500 = 1{,}25 \times 10^{4}$
Kis szám	$0{,}00125 = 1{,}25 \times 10^{-3}$
Két külön szerep	1,25: számjegyek • 10e: nagyságrend

A lényeges számjegyeket és a nagyságrendet külön írjuk le.

A lebegőpontos szám alapötlete

$$x = (-1)^s \cdot m \cdot b^e$$

s: előjel • m: szignifikandusz • b: alap • e: kitevő

A bináris pont után is vannak helyiértékek

1 / 2 1 / 4 1 / 8 1 / 16

$0{,}1_2 = 0{,}5_{10}$ • $0{,}01_2 = 0{,}25_{10}$

$0{,}101_2 = 1/2 + 1/8 = 0{,}625_{10}$

A tört helyiértékek: 2^{-1}, 2^{-2}, 2^{-3}, …

Alakítsuk át a 13,25-öt binárisra!

Egész rész	$13_{10} = 1101_2$
Tört rész	$0{,}25_{10} = 0{,}01_2$
Együtt	$13{,}25_{10} = 1101{,}01_2$

Ez a szám véges bináris törtként pontosan felírható.

Normalizálás: az érték marad, az alak változik

Kiinduló alak	$1101{,}01_2$
A pontot eltoljuk	$1{,}10101_2 \times 2^3$
Ellenőrzés	$1{,}65625 \times 8 = 13{,}25$

Normál bináris számnál $1 \le m < 2$, ezért az eleje mindig 1.

Kis számnál negatív kitevő jelenik meg

Kiinduló alak	$0{,}00101_2$
Normalizálva	$1{,}01_2 \times 2^{-3}$
Tízes ellenőrzés	$0{,}15625 = 1{,}25 / 8$

A negatív kitevő kis nagyságot jelent; nem negatív előjelet.

Melyik a normalizált bináris alak?

A $101{,}1_2$ számot hogyan írjuk $1 \le m < 2$ alakban?

A: $1{,}011_2 \times 2^2$

B: $10{,}11_2 \times 2^1$

C: $0{,}1011_2 \times 2^3$

Előbb gondolkodjunk, utána ellenőrizzük közösen!

Három mezővel írjuk le a számot

Előjelbit s

0: pozitív
1: negatív

Az érték előjelét adja.

Karakterisztika E

Az eltolt kitevő kódja.

A nagyságrendet adja.

Törtmező F

A szignifikandusz tört része.

A számjegyeket adja.

E a tárolt kód, e a valódi kitevő. A két jelölést külön használjuk.

IEEE 754: közös szabályok az ábrázoláshoz

1985

Az eredeti bináris szabvány.

2008

Kibővített szabvány: bináris és decimális formátumok.

2019

Átdolgozott kiadás, pontosításokkal.

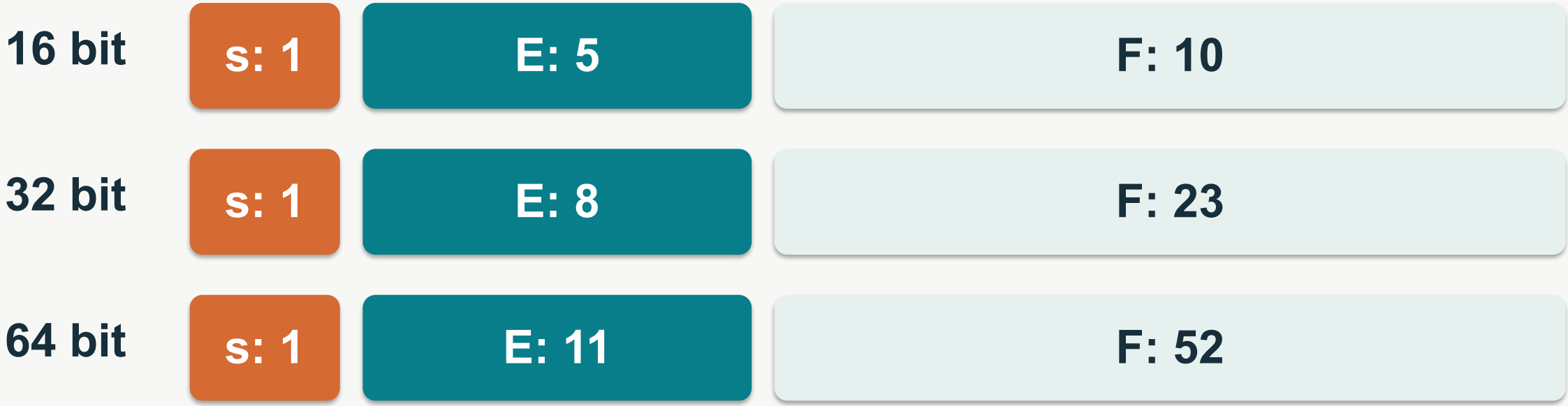
A szabvány a formátumok mellett a műveleteket és a kerekítést is meghatározza.

Feles, egyszeres és dupla pontosság

Formátum	Összes bit	s / E / F bitek	Normál pontosság
binary16 — feles	16	1 / 5 / 10	11 bináris számjegy
binary32 — egyszeres	32	1 / 8 / 23	24 bináris számjegy
binary64 — dupla	64	1 / 11 / 52	53 bináris számjegy

A normál szignifikanduszban a tárolt törtbitekhez egy implicit 1 is tartozik.

Ugyanaz a három mező, különböző bitkerettel



Több törtbit finomabb értékeket, több kitevőbit nagyobb tartományt ad.

Melyik pontosság mire lehet elég?

binary16

Kevés memória.
Kb. 3 tizedes
szignifikáns számjegy.

Grafika, egyes AI-adatok.

binary32

Köztes tárolóméret.
Kb. 7 tizedes
szignifikáns számjegy.

Sok mérési és grafikai adat.

binary64

Nagyobb tárolóméret.
Kb. 16 tizedes
szignifikáns számjegy.

Igényesebb számítások.

A számjegyszám közelítő jellemzés, nem a tizedesjegyek rögzített darabszáma.

A tartomány és a pontosság két külön kérdés

Formátum	Legnagyobb pozitív véges	1 utáni következő érték
binary16	65 504	$1 + 2^{-10}$
binary32	$\approx 3{,}40 \times 10^{38}$	$1 + 2^{-23}$
binary64	$\approx 1{,}80 \times 10^{308}$	$1 + 2^{-52}$

A nagy tartomány nem garantál sok pontos számjegyet egy adott nagyságrendben.

Eltolt karakterisztika: a kitevőhöz hozzáadunk egy állandót

$$E = e + B$$

E: tárolt karakterisztika • e: valódi kitevő • B: eltolás, azaz bias

A három formátum eltolása

Formátum	E bitszáma	B eltolás	Normál valódi kitevők
binary16	5	15	−14 … +15
binary32	8	127	−126 … +127
binary64	11	1023	−1022 … +1023

k kitevőbitnél $B = 2^{k-1} - 1$. A két szélső kód nem normál számot jelöl.

binary32 példa: kitevőből tárolt kód

Valódi e	E = e + 127	E bináris kódja
−3	124	01111100
0	127	01111111
+3	130	10000010

A 10000010 kód 130-at tárol, de a valódi kitevő csak 3.

Normál bináris érték: a mezők együttes jelentése

$$x = (-1)^s \cdot (1 + F / 2^t) \cdot 2^e$$

$e = E - B$ • t: a tárolt törtbitek száma • F: a törtmező egész értéke

A vezető 1-et nem kell külön tárolni

Teljes szignifikandusz	$1{,}10101_2$
Tárolt törtmező eleje	10101
binary32-ben kitöltve	10101000000000000000000

23 tárolt törtbit + 1 implicit vezető bit = 24 bites normál pontosság.

Kódoljuk a 13,25-öt binary32 formátumban!

Normalizált alak	$1{,}10101_2 \times 2^3$
Előjel és karakterisztika	$s = 0 \bullet E = 3 + 127 = 130$
Törtmező	F: 10101 és még 18 nulla

Először az értéket alakítjuk át, csak utána töltjük ki a mezőket.

A 13,25 teljes binary32 kódja

Előjel s	Karakterisztika E	Törtmező F
0	10000010	10101000000000000000000
1 bit	8 bit	23 bit

Hexadecimálisan: 0x41540000

Visszaellenőrzés: (+1) × 1,65625 × 2^3 = 13,25.

A −13,25-höz az előjelbit változik

Előjel s	Karakterisztika E	Törtmező F
1	10000010	10101000000000000000000
negatív	e = 3	ugyanaz a törtmező

Ez nem a teljes bitsorozat kettes komplemense: az s bit jelöli az előjelet.

Visszafejtés: 1 | 10001 | 0110000000

Előjel	$s = 1 \rightarrow$ negatív
Kitevő binary16-ban	$E = 17 \rightarrow e = 17 - 15 = 2$
Szignifikandusz és érték	$1{,}011_2 = 1{,}375 \rightarrow -1{,}375 \times 4 = -5{,}5$

A mezőméretből tudjuk, melyik formátumot kell értelmezni.

A tárolt kód még nem a valódi kitevő

binary32-ben E = 128. Mekkora a valódi e?

A: 128

B: 1

C: −1

Előbb gondolkodjunk, utána ellenőrizzük közösen!

Közös feladat: 5,75 kódolása binary16-ban

Először az érték

Írd át binárisra,
majd normalizáld!

$5{,}75_{10}$ = ?

Utána a mezők

s: 1 bit
E: 5 bit, bias 15
F: 10 törtbit

Kell-e kerekíteni?

Dolgozzunk párokban 3 percig, majd ellenőrizzük együtt!

Rövid szünet: eddig három mezőt fejtettünk meg

Előjel + nagyságrend + számjegyek

A folytatásban: mi történik, ha az érték nem fér el pontosan?

A tárolható számok között rések vannak



Szemléltető formátum: csak 3 tárolt törtbit.

Ha a pontos érték nincs a tárolható halmazban, kerekítenünk kell.

Nagyobb nagyságrendben nagyobb a lépés



1 és 2 között: 0,25-ös lépés

2 és 4 között: 0,50-es lépés

Szemléltető formátum, 2 tárolt törtbittel: nem egyenletes a teljes számvonal.

Elfér a szám, de a +1 már elveszhet

```
float x = 16_777_216f;
float y = x + 1f;

Console.WriteLine(y == x);
// True
```

2^{24} mellett

binary32-ben a következő nagyobb érték már 2-vel távolabb van.

A +1 félútra esik.

ULP: a szomszédos értékek helyi távolsága

Az 1 fölötti lépés binary32-ben: 2^{-23}

Ugyanez binary64-ben 2^{-52}. Más nagyságrendben más a lépés.

Miért különleges a 0,1?

Tízes alapban	$0{,}1_{10} = 1/10$
Kettes alapban	$0{,}0001100110011\ldots_2$
Véges tárolóban	A végtelen sorozatot kerekíteni kell.

A 0,1 véges decimális tört, de nem véges bináris tört.

A 0,1 + 0,2 meglepetése C#-ban

```
using System.Globalization;

double x = 0.1 + 0.2;
Console.WriteLine(x.ToString(
    "G17",
    CultureInfo.InvariantCulture));
Console.WriteLine(x == 0.3);
```

A kimenet

0.30000000000000004
False

A tárolt közelítések összege is kerekített.

A zárójelezés is változtathat az eredményen

```
double a = 1e16;
double b = -1e16;
double c = 1.0;

Console.WriteLine((a + b) + c);
// 1
Console.WriteLine(a + (b + c));
// 0
```

Nem asszociatív

b + c kerekítéskor ismét b lehet.

A kis tag hatása így elveszik.

Közelítő összehasonlítás: a hibahatár a feladat része

```
double a = 0.1 + 0.2;
double b = 0.3;
double tolerance = 1e-12;

bool close =
    Math.Abs(a - b) <= tolerance;
// True ebben a példában
```

Mekkora eltérés fér bele?

A példában kis, véges értékeket hasonlítunk.

A toleranciát a feladat és a skála alapján választjuk.

A karakterisztika szélső kódjai külön jelentést kapnak

E mező	F törtmező	Jelentés
Csupa 0	Csupa 0	+0 vagy −0
Csupa 0	Nem nulla	Szubnormál szám
Nem szélső kód	Bármely érték	Normál szám
Csupa 1	Csupa 0	+∞ vagy −∞
Csupa 1	Nem nulla	NaN

Az előjelbit a normál, szubnormál, nulla és végtelen érték előjelét adja.

Kétféle nulla: +0 és −0

Összehasonlítás

+0 == −0

Numerikusan egyenlők,
a bitmintájuk eltér.

Az előjelnek lehet hatása

1 / (+0) → +∞
1 / (−0) → −∞

Alapértelmezett lebegőpontos viselkedés.

Az előjeles nulla információt őrizhet a megközelítés irányáról.

A végtelen nem egy nagyon nagy véges szám

```
double zero = 0.0;
double inf = 1.0 / zero;

Console.WriteLine(
    double.IsPositiveInfinity(inf));
// True
```

+∞ és −∞

Jelölhetnek túl nagy eredményt vagy nem nulla érték nullával osztását.

A további műveleteknek is vannak szabályai.

NaN: nem számérték, nem egy szöveges hibaüzenet

```
double zero = 0.0;
double x = zero / zero;

Console.WriteLine(double.IsNaN(x));
// True
Console.WriteLine(x == x);
// False
```

Érvénytelen művelet

$0 / 0 \rightarrow \text{NaN}$

$\sqrt{(-1)} \rightarrow \text{NaN}$

Ellenőrzéshez IsNaN kell.

Öt IEEE jelzési helyzet

Jelzés	Szemléltető ok
Érvénytelen művelet	0 / 0, vagy valós $\sqrt{(-1)}$
Osztás nullával	Nem nulla, véges szám / nulla
Túlcsordulás	Túl nagy nagyságú véges eredmény
Alulcsordulás	Nagyon kis és pontatlan eredmény
Pontatlanság — inexact	A pontos eredményt kerekíteni kell

Az IEEE jelzés és a programozási nyelv kivétele külön fogalom.

Túlcsordulás: túl nagy az eredmény nagysága

```
double x = double.MaxValue;
double y = x * 2.0;

Console.WriteLine(
    double.IsPositiveInfinity(y));
// True
```

Kifelé a tartományból

Az eredmény már nem tárolható véges double-ként.

Ez eltér az egész számok körbefordulásától.

Alulcsordulás közelében: közeledünk a nullához

Még nem feltétlenül nulla

A legkisebb normál érték alatt szubnormál értékek is léteznek.

Csökkenhet a relatív pontosság.

Végül nulla lehet

Ha a szám ezeknél is túl kicsi, kerekítéskor nullát kaphatunk.

Az előjel megmaradhat.

A szubnormál számok fokozatos átmenetet adnak a normál számok és a nulla között.

Szubnormál szám: a vezető bit most 0

$$x = (-1)^s \cdot (F / 2^t) \cdot 2^{1-B}$$

E = 0 és F ≠ 0. Nincs implicit vezető 1; a kitevő 1 − B.

A legkisebb pozitív értékek kétféle határa

Formátum	Legkisebb normál	Legkisebb szubnormál
binary16	$2^{-14} \approx 6{,}10 \times 10^{-5}$	$2^{-24} \approx 5{,}96 \times 10^{-8}$
binary32	$2^{-126} \approx 1{,}18 \times 10^{-38}$	$2^{-149} \approx 1{,}40 \times 10^{-45}$
binary64	$2^{-1022} \approx 2{,}23 \times 10^{-308}$	$2^{-1074} \approx 4{,}94 \times 10^{-324}$

A „legkisebb pozitív szám” nem ugyanaz, mint a „legkisebb pozitív normál szám”.

binary16: ugyanaz a kis lépés a határ két oldalán

Legnagyobb szubnormál	1023×2^{-24}
Legkisebb normál	$1024 \times 2^{-24} = 2^{-14}$
A különbség	Mindössze 2^{-24}

A normál tartomány alá kerülve még maradnak különböző, nem nulla kódok.

A double.Epsilon nem általános hibahatár

```
double tiny = double.Epsilon;
double half = tiny / 2.0;

Console.WriteLine(half == 0.0);
// True
```

A név félrevezethet

double.Epsilon = 2^{-1074}

A legkisebb pozitív double;
nem az 1 melletti rés.

Kicsi eredmény: biztosan nulla?

A legkisebb pozitív normál binary32 értéket 2-vel osztjuk. Mit kaphatunk?

A: pontos szubnormált

B: csak nullát

C: pozitív végtelent

Előbb gondolkodjunk, utána ellenőrizzük közösen!

A kerekítés a tárolható szomszédot választja ki

Pontos eredmény → tárolható érték

Kerekítés előfordulhat beolvasáskor, konverziókor és egy művelet után is.

Négy alapvető bináris kerekítési mód

Mód	Döntés
Legközelebbi, félúton páros	A közelebbi érték; holtversenyben páros utolsó bit
Nulla felé	A kisebb abszolútértékű szomszéd
+∞ felé	A pontos eredménynél nem kisebb szomszéd
−∞ felé	A pontos eredménynél nem nagyobb szomszéd

Bináris eredmények alapértelmezett módja: legközelebbi, félúton páros.

Félúton páros: melyik bitet tartjuk meg?

Pontos érték, 3 törtbitre	$1{,}0101_2$
Két egyformán közeli érték	$1{,}010_2$ vagy $1{,}011_2$
Páros utolsó bitet választunk	$1{,}010_2$ — a végső bit 0

Csak a pontos félútnál dönt a paritás. Egyébként a közelebbi érték nyer.

A páros szabály máskor felfelé választ

Pontos érték, 3 törtbitre	$1{,}0111_2$
Két egyformán közeli érték	$1{,}011_2$ vagy $1{,}100_2$
Páros utolsó bitet választunk	$1{,}100_2$ — a végső bit 0

A félúton páros módszer csökkentheti az egyirányú kerekítési torzítást.

Kerekítési irányok: egyszerű egészre kerekítés

Mód	+2,7-ből	−2,7-ből
Legközelebbi	+3	−3
Nulla felé	+2	−2
+∞ felé	+3	−2
−∞ felé	+2	−3

A nulla felé és a −∞ felé kerekítés negatív számnál eltér.

Egy további félutas szabály: nullától elfelé

Legközelebbi, félúton páros

2,5 → 2
3,5 → 4
−2,5 → −2

A páros utolsó számjegy dönt.

Legközelebbi, félúton elfelé

2,5 → 3
3,5 → 4
−2,5 → −3

A nagyobb abszolútérték dönt.

Az IEEE roundTiesToAway szabálya bináris implementációban nem kötelező.

C#: a decimális kerekítés szabályát megadhatjuk

```
decimal x = 2.5m;

decimal even = Math.Round(x,
    MidpointRounding.ToEven);
decimal away = Math.Round(x,
    MidpointRounding.AwayFromZero);
// even: 2, away: 3
```

Tudatos döntés

A Math.Round alapértelmezése ToEven.

A szabályt a feladat követelménye határozza meg.

A szép kiírás nem változtatja meg a tárolt értéket

```
using System.Globalization;

double x = 0.1 + 0.2;
Console.WriteLine(x.ToString(
    "F2",
    CultureInfo.InvariantCulture));
// 0.30
```

Megjelenítés

A formázott szöveg két tizedesjegyű.

x továbbra is ugyanazt a double értéket tárolja.

Mit jelent a „félúton páros”?

Legközelebbi, félúton páros móddal egészre kerekítve mennyi a −2,5?

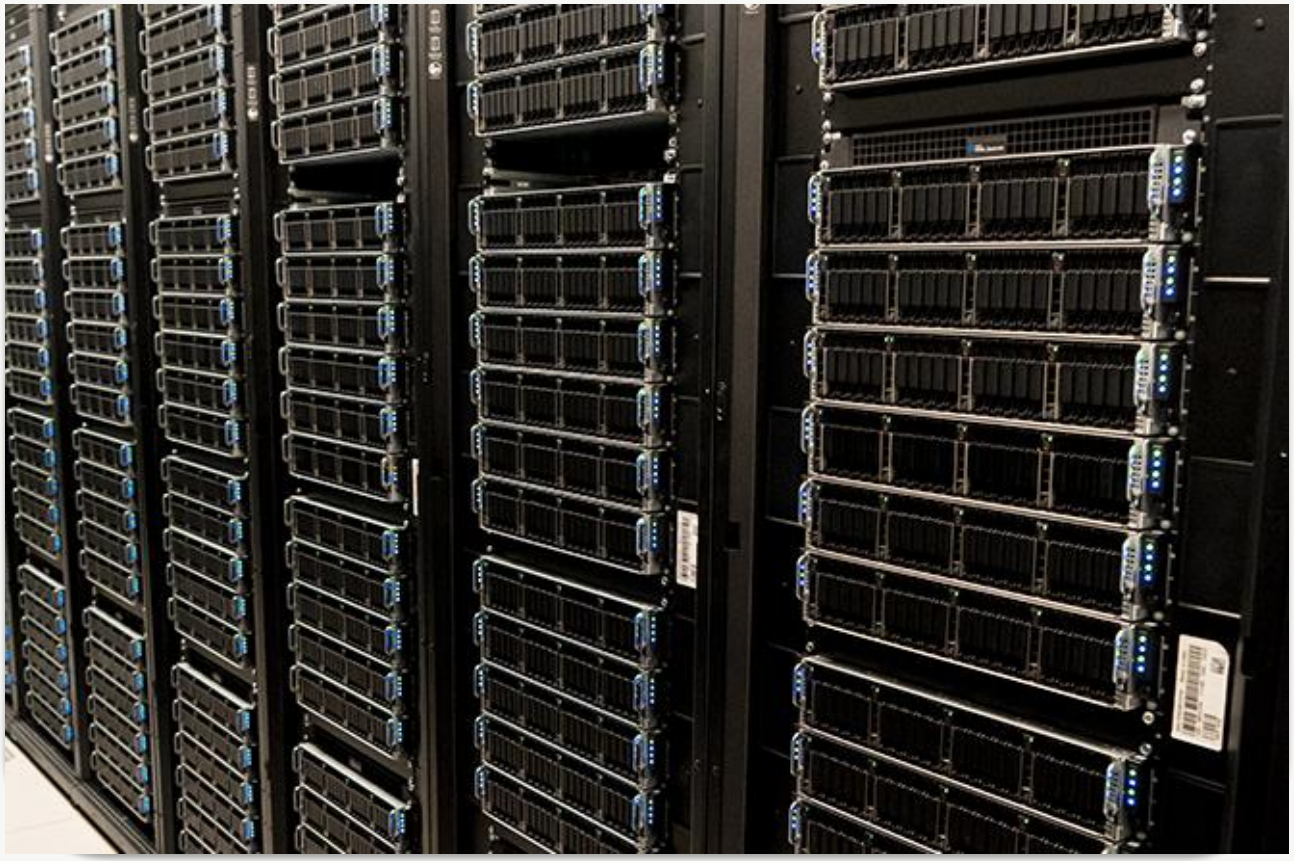
A: −3

B: −2

C: 0

Előbb gondolkodjunk, utána ellenőrizzük közösen!

FLOPS: lebegőpontos műveletek másodpercenként



FLOP/s = művelet / idő

Floating-point operations per second.

A számítási teljesítmény egy mérőszáma.

Fotó: Texas Advanced Computing Center / Frontera

Egy egyszerű teljesítménynyszámítás

Elvégzett műveletek	2 milliárd FLOP
Eltelt idő	0,5 másodperc
Átlagos sebesség	$2 \times 10^9 / 0{,}5 = 4$ GFLOP/s

Ez szemléltető mért adat, nem egy konkrét processzor teljesítményígérete.

FLOPS-többszörösök: decimális előtagok

Jelölés	Művelet másodpercenként
MFLOP/s	10^6 — millió
GFLOP/s	10^9 — milliárd
TFLOP/s	10^{12} — billió
PFLOP/s	10^{15} — billiárd
EFLOP/s	10^{18} — trillió

Ezek 1000-szeres lépések; nem bináris KiB/MiB típusú előtagok.

FMA: egy utasítás, két szokásosan számolt FLOP

Mit számol?

a × b + c

Szorzás és összeadás
egy összevont műveletben.

Miért érdekes?

Teljesítményszámolásnál
általában 2 FLOP.

IEEE FMA esetén csak
a végén van kerekítés.

A gépi utasítások darabszáma és az aritmetikai műveletszám nem ugyanaz.

Egy FLOPS-adat mellé három kérdés kell

Milyen pontosság?

FP16, FP32 vagy FP64?

Az eltérő számformátumok sebessége különbözhet.

Milyen mérés?

Elméleti csúcsteljesítmény vagy elért teljesítmény?

Melyik benchmark?

Milyen feladat?

A memória és a párhuzamos munka is számít.

A magas FLOPS nem jelent ugyanolyan gyors programot.

Összehasonlításhoz azonos pontosság és hasonló mérési feltételek szükségesek.

A három bináris típus C# / .NET környezetben

IEEE formátum	C# / .NET típus	Egyszerű példa
binary16	System.Half	Half h = (Half)1.5f;
binary32	float / System.Single	float f = 1.5f;
binary64	double / System.Double	double d = 1.5;

A suffix nélküli valós literál double; az f utótag float értéket ad.

És a decimal? Más reprezentáció, más feladat

Bináris float / double

0,1: bináris közelítés.

Nagy tartomány,
CPU/GPU számítások.

C# decimal

0,1m: pontosan tárolható.

128 bites decimális típus;
1/3 itt sem pontos.

A decimal nem az IEEE binary128, és nem tesz minden számítást pontossá.

Öt gondolat, amit érdemes hazavinni

Téma	Lényeg
Ábrázolás	Előjel, eltolt kitevő és szignifikandusz.
Normalizálás	Normál bináris számnál a vezető 1 implicit.
Pontosság	Nem minden értéknek van külön tárolható kódja.
Szélsőértékek	Szubnormál, ±0, ±∞ és NaN is létezik.
Művelet és sebesség	A kerekítés és a pontosság a FLOPS-adathoz is számít.

Kilépőkártya: három rövid ellenőrzés

Kódolás

Miért 130 a tárolt karakterisztika, ha binary32-ben e = 3?

Speciális érték

Mit jelent E = 0 és F ≠ 0?

Van-e implicit 1?

Kerekítés

A 2,5 egészre kerekítése miért lehet 2 vagy 3?

Mi dönti el?

A jó válasz megnevezi a formátumot és a kerekítési szabályt.

Források: IEEE 754 és a formátumok

IEEE Std 754-2019 — a szabvány hivatalos oldala

https://standards.ieee.org/ieee/754/6210/

IEEE munkacsoport — a 2008-as és 2019-es kiadás háttere

https://grouper.ieee.org/groups/msc/ANSI_IEEE-Std-754-2019/background/

Oracle Numerical Computation Guide — IEEE bináris formátumok

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_math.html

MathWorks — a feles pontosság felépítése

https://www.mathworks.com/help/coder/ug/what-is-half-precision.html

Oracle Numerical Computation Guide — fokozatos alulcsordulás

https://docs.oracle.com/cd/E60778_01/html/E60763/z4000ac020351.html

Források: C# és a számítás gyakorlata

Microsoft Learn — lebegőpontos típusok

https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

Microsoft Learn — System.Half

https://learn.microsoft.com/en-us/dotnet/api/system.half

Microsoft Learn — Math.Round

https://learn.microsoft.com/en-us/dotnet/api/system.math.round

Microsoft Learn — Double.Epsilon

https://learn.microsoft.com/en-us/dotnet/api/system.double.epsilon

NVIDIA — Floating Point and IEEE 754

https://docs.nvidia.com/cuda/floating-point/index.html

FLOPS, képek és további háttér

TOP500 — FLOP/s és benchmarkfogalmak

https://www.top500.org/resources/frequently-asked-questions/

Áramköri fotó: Alexandre Debiève / Unsplash

https://unsplash.com/photos/FO7JIlwjOtU

Szuperszámítógép-fotó: TACC / Frontera

https://tacc.utexas.edu/systems/all/

A képek internetről származnak; a bitábrák és számvonalak az órához készültek.